

Binomial Heaps - cont Lecture 9

Binomial heap is a set of binomial trees, each of which is heap ordered and at most 1 tree of order i

We know how to merge two heaps in $O(\log n)$ time where $n = \#$ elements in the two trees.

Insert an element a into a BH

Merge a single element heap consisting of a with BH in $O(\log n)$ time

Decrease key : The value of an element is reduced

Min : $\begin{cases} O(1) \text{ time if an explicit pointer is maintained} \\ \text{else } O(\log n) \text{ by inspecting all the roots of the binomial trees} \end{cases}$

Should be updated when it changes

Delete : First decrease value to $-\infty$
Then move it up to root node
Delete root node and create a Binomial heap on the children of the root node.

Then merge the two heaps

Extract Min : Delete the min (which is a root node) and follow the above procedure

Find the new min in the merged heap

Building a heap of n nodes

$O(n)$ using binary heaps

How about binomial heaps

Keep adding the elements one after the other

If it is $O(1)$ per insertion then it is $O(n)$

$O(\log n)$ for each insert then
 $\Rightarrow O(n \log n)$

To return

Consider an n bit counter

4 bit counter				<u>Cost</u>
B_3	B_2	B_1	B_0	
0	0	0	0	1
0	0	0	1	2
0	0	1	0	1
0	0	1	1	
0	1	0	0	
0	1	0	1	

Cost of changing
the count value
= # bits that
flips

Worst case is

0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	1	1	0
1	1	1	1

4

4 for 4 bits
 For an n
 bit counter,
 the worst case is
 n

The total cost of the counter going from
 0 to $2^n - 1 \leq n(2^n - 1)$

Observation: The j^{th} bit $0 \leq j \leq n-1$
 changes every 2^j steps
 $\Rightarrow \underline{2^n}$ overall

So total cost : $\sum_{j=0}^{n-1} \frac{2^n}{2^j} = \sum_j \frac{2^n}{2^j} \leq 2 \cdot 2^n$

Remark : The total cost is much less than worst-case $\times$ #steps

Binomial
heap
of
 n nodes

If we had $\log n$ list, then
total cost $\leq 2 \cdot 2^{\log n} = 2 \cdot n$

The cost of increment operation in B.H. is identical to the behavior of the counter and the number of list flips. Each $0 \rightarrow 1$ flip corresponds to incrementing the counter by 1.

going from B_i to B_{i+1} that takes
 $O(1)$ pointer ops and therefore the total cost
can be bounded by $O(n)$ for the n increments